Oppgave 1 – Modellering (25%, ~1 time)

Problembeskrivelse

Det er behov for en database for å håndtere passasjerer på flygninger. En flygning har avgang fra en flyplass på et klokkeslett, og har landing på en annen flyplass til et annet klokkeslett (vi trenger ikke å ta hensyn til forsinkelser her). En flygning har en unik kode, og blir utført av et bestemt flyselskap. En bestemt flygning skjer "hver" dag, og kan bare flyes en gang for dagen (men det kan være flere flygninger mellom samme flyplasser hvis de har forskjellig kode).

En person har navn, fødselsdato, telefonnummer og epostadresse.

En passasjer er en person som er med på en flygning. Hver flygning kan ha mange passasjerer (samme person kan selvsagt også være passasjer på mange flygninger). Det er nødvendig å lagre hvilken flygning og hvilken dag en person er passasjer.

(Denne problembeskrivelsen gjelder bare for denne oppgaven med mindre noe annet er nevnt.)

Oppgave

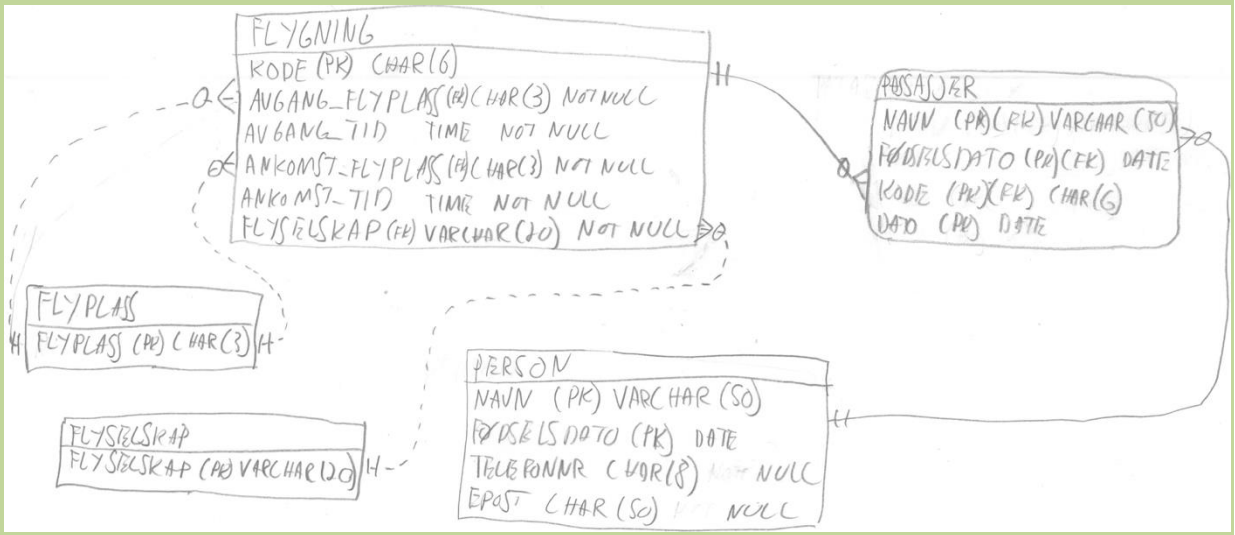
Lag (tegn) en logisk (fullstendig) ER-modell for denne databasen i en av notasjonene som har vært brukt i kurset.

Huskeliste

- Tabeller (som tilfredsstiller alt i problembeskrivelsen)
- Primærnøkler (for alle tabeller)
- Fremmednøkler (for alle relevante tabeller)
- Datatyper (for alle tabeller)
- Både min og maks kardinalitet (for alle forhold)
- Sterke/svake entitetstyper og eksistensavhengighet/uavhengighet (kråkefot-notasjon) eller eventuell type aggregering (UML-notasjon)
- Tilfredsstiller 1., 2., og 3. normalform (for alle tabeller)

Dersom du ikke er sikker på hvordan du skal løse en oppgave, forsøk å gjøre så mye som du kan!

Løsningsforslag



Vi starter med tabellen Flygning. Her tenker vi på en flygning som en daglig "flyrute" mellom to flyplasser. Som opplyst i problembeskrivelsen har den en unik kode, og følgelig bør dette da være en god PK. Dette betyr også at det blir en sterk entitet, siden den ikke arver del av PK fra en annen tabell. I løsningsforslaget er det tenkt at denne kan være noe slikt som SK1234, og datatypen blir dermed char(6), Andre format vil typisk også bli akseptert. Videre skal det være kolonner for avgang- og ankomst-flyplasser. Her har vi antatt at flyplasser kan identifiseres med koder som BGO, som gir datatype char(3). Igjen vil fullstendig flyplassnavn vil også bli akseptert. Vi har introdusert en verditabell for flyplass for å sikre at verdier blir konsistente. Dette vil bare være forventet for den øvre delen av karakterskalaen. Videre trengs det kolonner for avgang- og ankomst-tid. Her kan ikke datatype være stort annet enn time. Til slutt trengs det en kolonne for flyselskap. Her har vi bare valgt å bruke flyselskapets navn. Dette blir da en varchar(20). Også her har vi valgt å lage en verditabell, som først og fremst kan bli forventet i den øvre del av karakterskalaen. Vi kan merke oss et mulig normaliseringsproblem: vi har valgt et format for koden for flygningen som kan inneholde informasjon om flyselskapet – SK i SK1234. Vi velger å se bort fra denne problematikken her. Vi merker oss også at alle kolonner har not null. Dette fordi all informasjonen er nødvendig for at en flygning skal "gi mening". Vi må vite fra-/til-flyplass, tider og flyselskap.

Vi "hopper" så til tabellen Person. En person er ikke helt det samme som en passasjer slik vi har gjort det her: en person "blir" en passasjer når den er med på en bestemt flygning en bestemt dag. Vi får opplyst at en person skal ha navn. Ingenting er sagt om for-/etter-navn, så vi behøver ikke dele opp dette (men er heller ikke "feil" hvis det er gjort riktig). Vi antar at varchar (50) er en tilstrekkelig til å lagre et navn. Videre skal det være fødselsdato. Det er ingen gode grunner til å velge noe annet enn date her. Telefonnr skal også være med. For enkelhets skyld tenker vi at bare tar høyde for norske telefonnr, og char(8) kan dermed brukes. Det er selvsagt ikke noe i veien for å velge en varchar(n) for å også tillate utenlandske telefonnr. Til slutt trengs det epostadresse. Disse vil alltid ha forskjellig lengde, og en varchar(50) kan virke fornuftig. For denne tabellen er spørsmålet om PK litt mer komplisert.

Slik oppgaven er formulert, er det mulig å få inntrykk av at kombinasjonen av navn og fødselsdato vil være unik. Dette er nok en noe optimistisk antagelse. Muligens kan det argumenteres for at telefonnr eller epost enten aleine eller sammen med andre vil være en bedre PK. Men da roter man seg inn på spørsmålet om det er rimelig å "kreve" at alle passasjerer skal ha telefon og/eller epost. Og hva hvis flere passasjerer av en eller annen grunn har oppgitt samme telefonnr og/eller epost? Kanskje bør et løpennr vurders? Det blir fort mange diskutable vurderinger. De fleste "rimelige" valg vil bli akseptert (men noen kan gjerne bli vurdert som mer eller mindre gode enn andre). Kombinasjonen av navn og fødselsdato vil stort sett bli akseptert (men gjerne med en kommentar om at det er en "diskutabel" antagelse). Vi velger også å tillate null for telefonnr og epost i dette tilfellet. Igjen får vi her en sterk entitetstype siden den ikke arver del av PK fra en annen tabell.

Så kommer vi til den mest spennende delen av oppgaven: vi skal kople sammen flygning og person. Vi nevnte tidligere at vi tenker oss at når en person er med på en flygning en gitt dag, så blir dette en passasjer. En passasjer kan dermed bli tabellen som kobler sammen disse to tabellene. Hvis vi legger til grunn at en person er identifiser med navn og fødselsdato, og en flygning er identifisert med kode, så må vi i alle fall ha med disse tre kolonnene for tabellen Passasjer. Datatypene blir de samme, og de blir både PK og FK. Men: vi må også vite hvilken dag denne personen fløy. Vi må derfor også ha med en kolonne dato, som utvilsomt må ha datatypen date. Dette blir også en del av PK (men selvsagt ikke en FK). Vi får da en sammensatt PK med ikke mindre enn fire kolonner i dette tilfellet. (I praksis ønsker vi gjerne å unngå så kompliserte PK, men ifølge "boken" er det riktig.) Det er ikke direkte feil å introdusere et løpenr her, men det kan da oppstå problemstillinger rundt mulige duplikatverdier av samme passasjer. Dette blir i praksis en koblingstabell, og siden den arver deler av PK fra andre tabeller blir den en svak entitetstype, og får dermed avrundede hjørner. Som vanlig for en koblingstabell, må den ha nøyaktig en av hver av tabellene den kobler sammen. Videre vil en flygning kunne ha mange passasjerer, men kan i prinsippet ha ingen (tomt fly). En person kan være passasjer mange ganger. Man kan diskutere om det gir mening å legge inn en person i databasen uten at den er passasjer på minst en flygning. Det kan nok diskuteres, men vi tenker her at det er greit (det er heller ikke feil å si at det ikke gir mening).

Oppgave 2 – SQL (25%, ~1 time)

Problembeskrivelse

Oppgaven baserer seg løst på problembeskrivelsen til oppgave 1. Anta at en forenklet/ redusert/ minimal løsning kan bestå av disse to tabellene:

Flygning(flygning_kode, avgang_flyplass, avgang_klokkeslett)
Passasjer(navn, fødselsdato, flygning_kode*, flygning_dato, telefonnr);

Oppgave

a) Skriv SQL kode for å opprette tabellen Passasjer. Du kan anta at tabellen Flygning allerede eksisterer. Husk datatyper, primærnøkler, not/not null og eventuelle fremmednøkler med on update/delete
b) Skriv SQL kode for å opprette indeks på kolonnen telefonnr i tabellen Passasjer
c) Skriv SQL kode for å fjerne kolonnen telefonnr i tabellen Passasjer
d) Skriv SQL kode for å legge inn en ny passasjer. Du kan selv velge data
e) Skriv SQL kode for å finne alle passasjerer som er født 01.01.2000
f) Skriv SQL kode for å finne alle flygninger som har passasjerer født 01.01.2000
g) Skriv også forrige delspørsmål som relasjonsalgebra. Potensielt nyttige operatorer til å bruke i svar: π, σ, $\cup$, $\cap$, $\times$, $\bowtie$, $\ltimes$, $\rtimes$ og $\mapsto$ (du kan kopiere tegnene du trenger fra oppgaven inn i besvarelsen)
h) Skriv SQL kode som for hver dag viser hvor mange passasjerer som (totalt for alle flygninger) har flydd denne dagen
i) Skriv SQL kode som for hver flygning viser hvor mange passasjerer som (totalt for alle dager) har flydd denne flygningen
j) Skriv SQL-kode for å finne personer (passasjerer) som har flydd mer enn en gang samme dag (du kan anta at passasjerer med samme navn og fødselsdato er samme person)

Dersom du ikke er sikker på hvordan du skal løse en oppgave, forsøk å gjøre så mye som du kan!

Løsningsforslag

a) Create table passasjer(
navn varchar(50) not null,
fødselsdato date not null,
flygning_kode char(6) references flygning(flygning_kode)
on delete restrict on update cascade,
flygning_dato date not null,
telefonnr char(8) null,
primary key(navn, fødselsdato, flygning_kode, flygning_dato)
);
Andre "fornuftige" lengder for navn aksepteres. Fødselsdato bør være date. For flygning_kode aksepteres også bare numeriske verdier. Andre valg for on update/delete kan aksepteres hvis de gir mening. Datatyper for telefonnr som tillater utenlandske nummer er også greit.

b) Create index on passasjer(telefonnr);

c) Alter table passasjer drop column telefonnr;

d) Insert into passasjer
values ('Kari Nordmann', '01-01-2020', 'SK1234', '98765432');
Her er variant som setter alle verdier, kan også angi liste over kolonner som skal settes først.

e) Select * from passasjer where fødselsdato='2000-01-01';

f) Select distinct flygning_kode, flygning_dato from passasjer
where fødselsdato='2000-01-01';
(Denne oppgaven var egentlig tenkt å bli tolket på en mer komplisert måte som ville involvere en join, men legger nå til grunn en enklere tolkning. Andre fornuftige tolkninger blir også akseptert.)

g) $\pi_{\text{flygning_kode, flygning_dato}}(\sigma_{\text{fødselsdato='2000-01-01'}}(\text{passasjer}))$
Vi trenger ikke ta hensyn til distinct her. Litt variasjoner i notasjonen er akseptert. Svaret her er tenkt å være basert på det man skrev i forrige delspørsmål. (Endret tolkning av forrige delspørsmål påvirker også dette spørsmålet.)

h) Select flygning_dato, count(*) from passasjer group by flygning_dato;

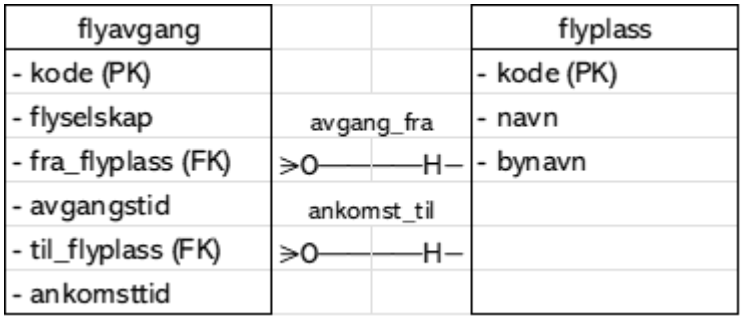
i) Select flygning_kode, count(*) from passasjer group by flygning_kode;

j) Select navn, fødselsdato, flygning_dato, count(*)
from passasjer
group by navn, fødselsdato, flygning_dato
having count(*) > 1;
Dette kan løses på forskjellige måter, for eksempel med indre spørring. Men dette er trolig den løsningen som er "enklest". Andre løsninger blir generelt akseptert dersom de gir samme resultat.

Oppgave 3 (25% ~ 1 time) – ORM/JPA

Vi skal jobbe litt med en database for Avinor. Databasen har informasjon om norske **flyavgang**er (flights) og **flyplass**er.

Her er den logiske ER-modellen for databasen:



Som du ser, er det **to en-til-mange-forhold** mellom flyavgang og flyplass, i betydningen at hver flyavgang har nøyaktig én **fra_flyplass** og én **til_flyplass**, mens en flyplass (i en av disse to rollene) kan være knyttet til mange ulike flyavganger.

PK angir primærnøkkel, og **FK** angir evt. fremmednøkler.

Her er tilsvarende tabeller med noen eksempeldata:

flyavgang

kode (PK)	flyselskap	fra_flyplass (FK)	avgangstid	til_flyplass (FK)	ankomsttid
...					
WF170	Widerøe	BGO	13:50	OSL	14:40
SK4164	SAS	BGO	13:50	TRD	14:55
SK269	SAS	OSL	14:35	BGO	15:25
...					

flyplass

kode (PK)	navn	bynavn
...		
BGO	Bergen lufthavn, Flesland	Bergen
OSL	Oslo lufthavn, Gardermoen	Oslo
TRD	Trondheim lufthavn, Værnes	Trondheim
...		

Alle data er av typen VARCHAR, med unntak av avgangstid og ankomsttid, som er av typen TIME.

Vi ønsker å jobbe med denne databasen i et Java-program.

a) (8% ~ 20 min) Skriv Java-klassene for entitetstypene `Flyavgang` og `Flyplass` som tilsvarer tabellene i eksempelet vist over. Du trenger ikke å skrive metoder, kun instans-/objektvariabler og JPA-annoteringer. Husk de nødvendige annoteringer for å angi forholdet mellom entitetene:
   - Vi **ønsker** at en **Flyplass** skal ha en **liste over flyavganger fra denne**, slik at vi ved behov enkelt kan få ut f.eks. alle avganger fra Bergen lufthavn, Flesland.
   - Vi ønsker **IKKE** at en **Flyplass** skal ha en **liste over ankomster til denne**. Hvis det er behov for en liste over ankomster, må dette hentes via en spørring.

```
Løsningsforslag m/merknader:

1  @Entity
   public class Flyavgang {

1      @Id private String kode;

private String flyselskap;

1      private LocalTime avgangstid;
private LocalTime ankomsttid;

1      @ManyToOne @JoinColumn(name = "fra_flyplass")
1      private Flyplass fraFlyplass;

1      @ManyToOne @JoinColumn(name = "til_flyplass")
1      private Flyplass tilFlyplass;
...
   }

   @Entity
   public class Flyplass {

@Id private String kode;
private String navn;
private String bynavn;

1      @OneToMany(mappedBy = "fraFlyplass", fetch = FetchType.EAGER)
2      private List<Flyavgang> flyavganger;
       ...
   }
```

Du kan i de videre oppgavene anta at entitetsklassene inneholder de nødvendige konstruktører, gettere og settere, etc. du trenger i løsningene dine.

Vi antar at vi har en hjelpeklasse **`FlyavgangDAO`** for databaseoperasjoner. Du skal lage et par metoder i denne. Du kan anta at en EntityManagerFactory kalt emf er opprettet og tilgjengelig for bruk i metodene.

b) (5% ~ 10 min) Skriv en metode

```
Flyplass finnFlyplass(String flyplasskode)
```

i FlyavgangDAO som henter ut flyplassen med gitt flyplasskode, f.eks. "**BGO**". Hvis ingen flyplasser med denne flyplasskoden finnes, skal det returneres null.

```
Løsningsforslag m/merknader:

    public Flyplass finnFlyplass(String flyplasskode) {

1       EntityManager em = emf.createEntityManager();
1       try {
5          return em.find(Flyplass.class, flyplasskode);
1       } finally {
2          em.close();
        }
    }

Alternativt, med try-with-resources:

    public Flyplass finnFlyplass(String flyplasskode) {
5       try (EntityManager em = emf.createEntityManager()) {
5          return em.find(Flyplass.class, flyplasskode);
        }
    }
```

c) (6% ~ 15 min) Skriv en metode

```
List<Flyavgang> finnFlyavgangerMellom(String byFra, String byTil)
```

i FlyavgangDAO som henter ut alle flyavganger mellom to byer (altså mellom flyplasser tilhørende by, oppgitt med **bynavn**), f.eks. alle flyavganger fra "**Bergen**" til "**Oslo**. Tips: Her må det skrives en JPQL-spørring.

```
Løsningsforslag m/merknader:

    public List<Flyavgang> finnFlyavgangerMellom(String byFra, String byTil) {

3      String q = """
             select flight from Flyavgang as flight
where  flight.fraFlyplass.bynavn = :byFra
and    flight.tilFlyplass.bynavn = :byTil
order by flight.avgangstid asc """; //Ikke en del av oppgaven

        EntityManager em = emf.createEntityManager();
        try {
2            TypedQuery<Flyavgang> query = em.createQuery(q, Flyavgang.class);
2            query.setParameter("byFra", byFra);
1            query.setParameter("byTil", byTil);
2            return query.getResultList();

        } finally {
            em.close();
        }
    }

Alternativt, med try-with-resources og method chaining:

    public List<Flyavgang> finnFlyavgangerMellom(String byFra, String byTil) {

3       String q = ...

        try (EntityManager em = emf.createEntityManager()) {
2            return em.createQuery(q, Flyavgang.class)
2                   .setParameter("byFra", byFra)
1                   .setParameter("byTil", byTil)
2                   .getResultList();
         }
    }
```

d) (6% ~ 15 min) Skriv en metode

```
boolean endreTiderForFlyavgang(String flightkode,
        LocalTime nyAvgangstid, LocalTime nyAnkomsttid)
```

i FlyavgangDAO som endrer avgangstid og ankomsttid for en gitt flyavgang (flight). Hvis oppdateringen går greit skal det returneres **true**. Hvis oppgitt kode for flyavgangen (flightkode) ikke finnes i databasen, skal det returneres **false**.

```
Løsningsforslag m/merknader (med try-with-resources):

    public boolean endreTiderForFlyavgang(String flightkode,
            LocalTime nyAvgangstid, LocalTime nyAnkomsttid) {

        try (EntityManager em = emf.createEntityManager()) {
            EntityTransaction tx = em.getTransaction();

1           tx.begin();

2           Flyavgang flight = em.find(Flyavgang.class, flightkode);

1           if (flight == null) {
                return false;
            }

2           flight.setAvgangstid(nyAvgangstid);
2           flight.setAnkomsttid(nyAnkomsttid);

1           tx.commit();
1           return true;
        }
    }
```

```
Sensormerknader:

En del besvarelser gjorde en em.persist(), enten på eksisterende
      eller på nyopprettet flyavgang. Dette blir feil uansett, da en
      rad for denne flyavgangen allerede finnes i databasen. Det vil bli
      kastet et DuplicateKeyException. Det trekkes poeng for dette.

      En del besvarelser gjorde en unødvendig em.merge() på managed
      flyavgang funnet med em.find(). Det gis ikke trekk for dette.

      Noen besvarelser opprettet en ny flyavgang med flightkode,
      nyAvgangstid og nyAnkomsttid, og gjorde en em.merge() på denne.
I så fall blir eksisterende flyavgang erstattet med en ufullstendig
      ny flyavgang (mangler flyselskap, fra_flyplass og til_flyplass).
      Det trekkes poeng for dette.
```

Oppgave 4 – XML, JSON og NoSQL (25%, ~1 time)

4.1 XML og XPath (13% ~ 30 min)

I oppgavene under (a-f) skal du generere Xpath uttrykk basert på XML dokumenet i vedlegg 4.1 (biler.xml).

Eksempel:
Oppgave: Skriv et XPath utrykk for å finne alle bilmerker i XML filen.
Svar: /biler/bil/merke/text()

Tips:
Når oppgaven inneholder teksten "all informasjon om alle biler" betyr at du skal hente de aktuelle elementene for de bilene som omfattes av spørringen.

For hver av Xpath uttrykkene gies det score fra 0-10.

Vi benytter hele skalaen fra 0 til 10, og man trenger ikke å ha alt rett for å få en score.

Hvis man f.eks. på oppgave a) har svart "/biler/bil/text()", så gies kun 8 poeng.
Metoden text() benyttes for å hente tekst data for et element. Elementet <bil> har ikke tekst, men har sub-elementer, og dermed er det ikke riktig å benytte metoden "text()".

a) Skriv et Xpath uttrykk for å hente alle biler.
   Tips! Husk at du skal finne "alle biler" (dvs bil "elementer").

/biler/bil

//bil

Godkjenner også:

/biler og //biler

b) Skriv et Xpath uttrykk for å finne alle registreringsnumre ("regnr").

/biler/bil/@regnr //bil/@regnr //@regnr

c) Skriv et Xpath uttrykk for å finne all informasjon om alle biler i klassen "SUV".

Vi har lært:

/biler/bil[klasse="SUV"] //bil[klasse="SUV"]

Godkjenner også:

//bil/klasse[contains(text(),"SUV")]/.. /biler/bil/klasse[text() = "SUV"]/..

d) Skriv et Xpath uttrykk for å finne all informasjon om alle biler produsert i 2018 eller senere.

Vi har lært:

/biler/bil[@produksjonsår>=2018]

//bil[@produksjonsår>2017]

e) Skriv et Xpath uttrykk for å finne all informasjon om alle biler med alarm.

Vi har lært:
/biler/bil/utstyr/alarm/../..

Andre løsninger som også godkjennes:
//bil[./utstyr/alarm]

/biler/bil[count(utstyr/alarm)>0]

f) Skriv et Xpath uttrykk for å finne all informasjon om alle biler med adaptive cruise-kontroll.

Vi har lært:
/biler/bil/utstyr/cruise-kontroll[@type="adaptiv"]/../..

Andre løsninger som også godkjennes:

//bil[./utstyr/cruise-kontroll/@type="adaptiv"]

//bil[./utstyr/cruise-kontroll[@type="adaptiv"]]

4.2 MongoDB Query API (6% ~ 15 min)

Firmaet «Lekre Hus AS» har opprettet to samlinger (Eng: «collections») «ansatte» og «kunder» i en MongoDB database.

Eksempel oppgave:
Gitt at en kunde ikke finnes fra før i samlingen «kunder», hva blir resultatet/output av følgende spørring?

```
> db.kunder.insertOne({_id: 1, kunde: "Per Hansen" })
```

Svar:
{ acknowledged: true, insertedId: 1 }

NB! I oppgavene under (a og b) skal du fylle inn verdier for «?», og svare på følgende format:

```
{
  acknowledged: true,
  insertedId: ?,
  matchedCount: ?,
  modifiedCount: ?,
  upsertedCount: ?
}
```

En utvikler skriver følgende MongoDB spørring:

```
> db.ansatte.updateOne(
    { _id: 1 },
    {
      $set: {
              fornavn: "Ola",
```

```
            etternavn: "Nordmann",
            oppdatert: new ISODate()

          },
      $setOnInsert: {opprettet: new ISODate() },
    },
    { upsert: true }
  )
```

For hver del oppgave (a, b og c) gies det score fra 0-10.

a) (2% ~ 5 min) Gitt at en ansatt **ikke finnes** fra før i samlingen "ansatte", hva blir resultatet av spørring over (svar på format vist over)?

```
{
  acknowledged: true,                                    2
  insertedId: 1,                                         2
  matchedCount: 0,                                       2
  modifiedCount: 0,                                      2
  upsertedCount: 1                                       2
}
```

b) (2% ~ 5 min) Gitt at en ansatt **finnes** fra før i samlingen "ansatte", hva blir resultatet av spørringen over (svar på format vist over)?

```
{
  acknowledged: true,                                    2
  insertedId: null,                                      2
  matchedCount: 1,                                       2
  modifiedCount: 1,                                      2
  upsertedCount: 0                                       2
}
```

c) (2% ~ 5 min) Gitt at ansatte (representert ved _id: 1) **ikke finnes** og spørringen over kjøres to ganger etter hverandre (med et minutts mellomrom), hva blir resultatet hvis vi kjører følgende spørring? (Bruk dagens dato og et valgfritt klokkeslett)
Skriv JSON dokumentet som returneres.

```
> db.ansatte.find({_id: 1})

[
  {                                                          2
    _id: 1,                                                  2
    etternavn: 'Nordmann',                                   2
    fornavn: 'Ola',
    oppdatert: ISODate('2025-05-17T10:21:00.000Z'),          2
    opprettet: ISODate('2025-05-17T10:22:00.000Z')           2
  }
]
```

Her trenger man ikke ha med array Det er kun json objektet {} med innhold (nøkkel/verdi par) som gir poeng.

4.3 MongoDB & Java (6% ~ 15 min)

En av utviklerne har skrevet følgende kode for å oppdatere informasjonen om en ansatt i klassen «AnsattRepository»:

```
import static com.mongodb.client.model.Filters.*;
import com.mongodb.client.model.UpdateOptions;
import com.mongodb.client.model.Updates;
import java.util.*;
...
public class AnsattRepository {
  private MongoCollection<Ansatt> collAnsatt = ...;

    …

  public void update(Ansatt ansatt) {
    Bson filter = Filters.eq("_id", ansatt.getId());

    Bson oppdateringer =
        Updates.combine(
          Updates.set("fornavn", ansatt.getFornavn()),
          Updates.set("etternavn", ansatt.getEtternavn()),
          Updates.set("stilling", ansatt.getStilling()),
          Updates.set("oppdatert", LocalDateTime.now()),
          Updates.setOnInsert("opprettet", LocalDateTime.now())
    );

    collAnsatt.updateOne(filter, oppdateringer);
  }
  ...
}
```

For hver del oppgave (a, b og c) gies det score fra 0-10.

a) (2% ~ 5 min) I koden over har utvikleren gjort en feil. Kodelinjen som inneholder "Updates.setOnInsert" vil aldri bli utført. Hvorfor? Og hva må til for koden skal fungere slik som utvikleren her har ment? (*Tips! Klassen "UpdateOptions")*

(4) Default setting for opsjonen "upsert" er "false". Dermed så vil denne oppdateringen aldri utføre en INSERT.

Vi må legge til en Options.upsert(true):

```
    public void update(Ansatt ansatt) {
       Bson filter = Filters.eq("_id", ansatt.getId());
(4)    UpdateOptions options = Options.upsert(true);

       Bson oppdateringer =
       Updates.combine(
          Updates.set("fornavn", ansatt.getFornavn()),
          Updates.set("etternavn", ansatt.getEtternavn()),
          Updates.set("stilling", ansatt.getStilling()),
          Updates.set("oppdatert", LocalDateTime.now()),
          Updates.setOnInsert("opprettet", LocalDateTime.now())
       );

(2)    collAnsatt.updateOne(filter, oppdateringer, options);

    }
```

I denne oppgaven holder det at man tar med følgende kodelinjer:

UpdateOptions options = Options.upsert(true);

collAnsatt.updateOne(filter, oppdateringer, **options**);

b) (2% ~ 5 min) Fullfør "create" metoden under for å legge til en ny ansatt.
Sørg for at verdien for nøkkelen "opprettet" alltid blir satt til tidspunkt (dato og klokkeslett) når metoden blir kjørt.
Tips! Bruk MongoCollection<Ansatt> variabelen "collAnsatt" variabelen. Anta at det er en metode "setOpprettet(LocalDateTime opprettetTid) i Ansatt klassen.

```
public void create(Ansatt ansatt) {

-- (2)
   ansatt.setOpprettet(LocalDateTime.now());
-- (2)         (4)        (2)
   collAnsatt.insertOne(ansatt);
}
```

c) (2% ~ 5 min) Fullfør "read" metoden under (som skal tilby søk etter ansatte).
Metoden skal ta et "Bson" filter som inn-parameter, og returnere en liste av ansatte.
Tips! Bruk "collAnsatt" variabelen, og metoden "find(…)" som returnerer en "FindIterator<Ansatt>".

```
public List<Ansatt> read(Bson filter) {
--  (2)
   List<Ansatt> liste = new ArrayList();

--  (2)            (2)      (2)
   colAnsatt.find(filter).into(liste);

--          (2)
   return liste;
}
```

Vedlegg 4.1: (Filnavn: biler.xml)


```xml
<?xml version="1.0" encoding="UTF-8"?>
<biler>
  <bil regnr="XK12345" produksjonsår="2016">
    <merke>Honda</merke>
    <model>Civik</model>
    <klasse>sedan</klasse>
    <detaljer>
      <seter>2</seter>
      <motor hestekrefter=="100" drift="FWD" ccm3="2000"/>
    </detaljer>
    <utstyr>
      <alarm/><keyless/><cruise-kontroll type="adaptiv"/>
    </utstyr>
  </bil>
  <bil regnr="ZZ22233" produksjonsår="2020">
    <merke>Toyota</merke>
    <model>Land Cruiser</model>
    <klasse>SUV</klasse>
    <detaljer>
      <seter>2</seter>
      <motor hestekrefter=="280" drift="AWD" ccm3="3000"/>
    </detaljer>
    <utstyr>
      <keyless/><cruise-kontroll type="manuell"/><ABS/>
    </utstyr>
  </bil>
  <bil regnr="FP89959" produksjonsår="1989">
    <merke>Porsche</merke>
    <model>959</model>
    <klasse>sportsbil</klasse>
    <detaljer>
      <seter>4</seter>
      <motor hestekrefter=="530" drift="AWD" ccm3="2850"/>
    </detaljer>
    <utstyr>
      <alarm/><cruise-kontroll type="manuell"/><turbo/>
    </utstyr>
  </bil>
</biler>
```